Computational Economics

Lecture: Q Functions, Optimality, and Value Function Iteration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lecture Overview

- **From a Fixed Policy to Policy Improvement**
  - State values, state-action values, and the maze example
  - Greedy choices and the policy improvement theorem
- **Optimality and Policy Iteration**
  - Bellman optimality and the one-step deviation principle
  - Exact policy evaluation and improvement
- **Value Function Iteration**
  - Contraction, stopping errors, and policy recovery
  - McCall job search and continuous saving

Recap: Computing the Value Function of a Given Policy

Consider a specific deterministic policy σ.

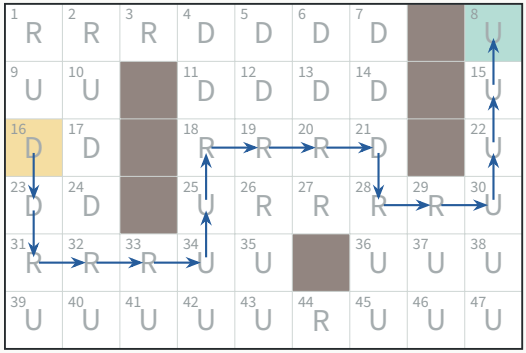


Figure 1: Actions and path under σ.

- U: up; D: down;
  L: left; R: right.
- Each move gives -1, including entry into cell 8.
- Set $\beta = 1$ for simplicity.
- Following this policy reaches cell 8 from every free cell.

Recap: Computing the Value Function of a Given Policy

With $\beta = 1$, compute values backward from cell 8, following the actions of σ.

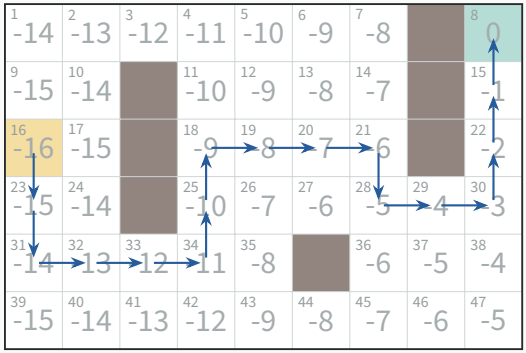


Figure 2: Values and path under σ.

$V^\sigma(8) = 0$. For every other cell,

$$V^\sigma(s) = -1 + V^\sigma(s').$$

$$V^\sigma(15) = -1 + 0 = -1,$$
$$V^\sigma(22) = -1 + (-1) = -2,$$
$$V^\sigma(30) = -1 + (-2) = -3.$$

Continuing backward gives $V^\sigma(16) = -16$.

Each value is minus the number of moves to cell 8 under σ.

From State Values to Action Values

The **state value function** $V^\sigma(s)$: follow the deterministic policy σ now and afterward.

$$V^\sigma(s) = \underbrace{r\big(s, \sigma(s)\big)}_{\text{Immediate reward}} + \underbrace{\beta\, \mathbb{E}[V^\sigma(s') \mid s,\ a = \sigma(s)]}_{\text{Discounted continuation value}}.$$

The **state-action value function** $Q^\sigma(s, a)$: choose a now, then follow σ.

$$Q^\sigma(s, a) = \underbrace{r(s, a)}_{\text{Immediate reward}} + \underbrace{\beta\, \mathbb{E}[V^\sigma(s') \mid s, a]}_{\text{Discounted continuation value}}.$$

The first action a is deliberately chosen; it need not be an action that σ selects at s.

Linking Q and V

Suppose the first action a is also chosen according to σ. Then the policy governs the entire rollout.

- **Deterministic policy:** $a = \sigma(s)$, so

$$V^{\sigma}(s) = Q^{\sigma}(s, \sigma(s)).$$

- **Randomized policy:** $a \sim \sigma(s)$. For a finite action set,

$$V^{\sigma}(s) = \sum_{a \in \mathcal{A}(s)} \Pr\{\sigma(s) = a\}\, Q^{\sigma}(s, a).$$

Example: Comparing First Actions

At cell 25, $\sigma(25) = \mathrm{U}$. Compare U and R, then follow σ. Set $\beta = 1$.

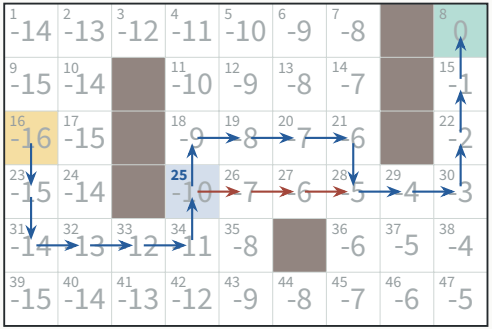


Figure 3: Policy path and shortcut from cell 25.

Up to cell 18:

$$Q^{\sigma}(25, \mathrm{U}) = -1 - 9 = -10.$$

Right to cell 26:

$$Q^{\sigma}(25, \mathrm{R}) = -1 - 7 = -8.$$

The policy chooses U, so $V^{\sigma}(25) = -10$.
Choosing R first gives a higher return.

Greedy Policy Improvement

For the finite-state results, assume finite feasible action sets, bounded rewards, $0 < \beta < 1$, and time-homogeneous dynamics. Policies are stationary and deterministic.

Given the exact value V^σ, choose a **greedy policy** at every state:

$$\widetilde{\sigma}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^\sigma(s, a).$$

Because the old action remains feasible,

$$Q^\sigma(s, \widetilde{\sigma}(s)) \geq Q^\sigma(s, \sigma(s)) = V^\sigma(s), \qquad \forall s.$$

Greedy choices maximize immediate reward **plus continuation value**, rather than immediate reward alone.

Policy improvement theorem: if the displayed inequality holds at every state, then

$$V^{\widetilde{\sigma}}(s) \geq V^\sigma(s), \qquad \forall s.$$

Why Policy Improvement Works

For any candidate continuation value v, the **policy evaluation operator** for $\widetilde{\sigma}$ is

$$(T_{\widetilde{\sigma}} v)(s) = r(s, \widetilde{\sigma}(s)) + \beta \mathbb{E}[v(s') \mid s, a = \widetilde{\sigma}(s)].$$

It is **monotone**: $v \leq w$ implies $T_{\widetilde{\sigma}} v \leq T_{\widetilde{\sigma}} w$, state by state.

The improvement condition is $V^{\sigma} \leq T_{\widetilde{\sigma}} V^{\sigma}$. Applying the same operator repeatedly gives

$$V^{\sigma} \leq T_{\widetilde{\sigma}} V^{\sigma} \leq T_{\widetilde{\sigma}}^2 V^{\sigma} \leq \cdots \longrightarrow V^{\widetilde{\sigma}}.$$

- Each update follows the new policy for one more step; the influence of the initial continuation vanishes at rate β^n.
- A strict improvement in the first inequality at a state gives a strict value gain at that state.

The Principle of Optimality

Define the **optimal values** over feasible policies:

$$V^*(s) = \max_\sigma V^\sigma(s), \qquad Q^*(s,a) = \max_\sigma Q^\sigma(s,a).$$

In this setting, a stationary deterministic policy is optimal among all admissible policies.

Principle of optimality: an optimal policy must have an optimal continuation from every state reached with positive probability.

$$Q^*(s,a) = r(s,a) + \beta \mathbb{E}[V^*(s') \mid s,a],$$
$$V^*(s) = \max_{a \in \mathcal{A}(s)} Q^*(s,a).$$

Combining them gives the **Bellman optimality equation**:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V^*(s') \mid s,a]\}.$$

Optimality as a One-Step Look Ahead

$$V^*(s) = \max_a Q^*(s, a)$$

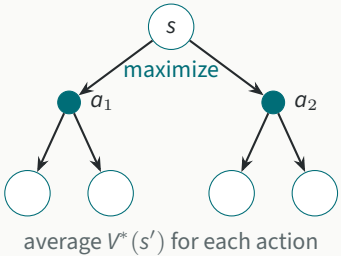


Figure 4: Choose now, average outcomes.

$$Q^*(s, a) = r(s, a) + \beta \mathbb{E}[\max_{a'} Q^*(s', a') \mid s, a]$$

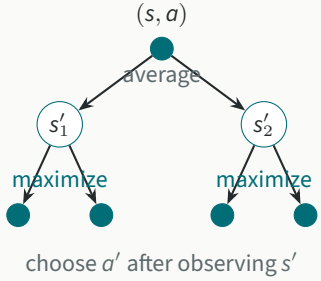


Figure 5: Fix now, choose at the next state.

Open circles: states. Filled circles: state–action pairs. Adapted from Sutton and Barto (2018), Figure 3.4; branches show possible next states.

The Bellman Optimality Operator

Replace the optimal continuation by a candidate v. The **Bellman optimality operator** is

$$(Tv)(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a) v(s') \right\}.$$

- $T_\sigma v$ uses the prescribed action $\sigma(s)$; Tv chooses the best action for continuation v.
- A policy greedy with respect to v attains this maximum: $T_\sigma v = Tv$.
- The optimal value satisfies $TV^* = V^*$: it is a **fixed point** of T.

Contraction and Uniqueness

Use $\|v - w\|_\infty = \max_s |v(s) - w(s)|$. For the same state and action,

$$|\beta \mathbb{E}[v(s') - w(s') \mid s, a]| \leq \beta \|v - w\|_\infty.$$

Taking maxima cannot enlarge this bound, since

$$|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|.$$

Therefore T is a **contraction**:

$$\|Tv - Tw\|_\infty \leq \beta \|v - w\|_\infty.$$

The contraction mapping theorem gives a unique fixed point, V^*, and convergence to it from any initial value vector. The same bound holds for each fixed-policy operator T_σ.

When Is a Policy Optimal?

The **one-step deviation principle** gives a test using the policy's own value:

$$\sigma(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^{\sigma}(s,a) \quad \forall s \quad \Longleftrightarrow \quad V^{\sigma} = V^{*}.$$

- **Sufficiency:** if no first-action change improves the return at any state, then

  $$V^{\sigma} = T_{\sigma} V^{\sigma} = T V^{\sigma}.$$

  The Bellman fixed point is unique, so $V^{\sigma} = V^{*}$.

- **Necessity:** if $V^{\sigma} = V^{*}$, then

  $$T_{\sigma} V^{*} = V^{*} = T V^{*},$$

  so σ attains the maximum at every state.

The comparison checks **all feasible actions at every state**, with the exact continuation value.

Policy Iteration

Start from any feasible policy σ_0. Repeat:

1. **Policy evaluation:** solve exactly for the current policy's value,

$$V^{\sigma_k} = T_{\sigma_k} V^{\sigma_k}.$$

2. **Policy improvement:** choose

$$\sigma_{k+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^{\sigma_k}(s, a), \qquad \forall s.$$

   Keep the current action whenever it is a maximizer.

3. If $\sigma_{k+1} = \sigma_k$, stop: the policy is optimal. Otherwise evaluate the new policy and repeat.

$$V^{\sigma_0} \leq V^{\sigma_1} \leq V^{\sigma_2} \leq \cdots \leq V^*.$$

Every nonoptimal policy is strictly improved at some state. There are finitely many deterministic policies, so the algorithm reaches an optimal policy in finitely many iterations.

Value Function Iteration

VFI updates a value estimate v_k, without evaluating a policy to completion.

1. Initialize v_0, for example at zero.
2. Using the old values, compute the scores and the new values:

$$Q_k(s,a) = r(s,a) + \beta \mathbb{E}[v_k(s') \mid s,a],$$
$$v_{k+1}(s) = \max_{a \in \mathcal{A}(s)} Q_k(s,a) = (Tv_k)(s).$$

3. Repeat these updates until the required accuracy is reached.

A **sweep** updates all states using the same old v_k; store the results in a separate array.

Contraction gives

$$\|v_k - V^*\|_\infty \leq \beta^k \|v_0 - V^*\|_\infty \longrightarrow 0.$$

v_k need not equal any V^σ, and Q_k need not equal any Q^σ. VFI convergence follows from contraction, not successive exact policy evaluations.

Example: Values Move Back Through the Maze

Consider cells 30, 22, and 15 near the goal (8) in the same maze. Set $\beta = 0.9$, $v_0 = 0$; each move before the goal costs one.

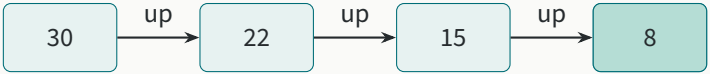


Figure 6: The final corridor to the goal.

	Cell 30	Cell 22	Cell 15	Cell 8
v_0	0	0	0	0
v_1	-1	-1	-1	0
v_2	-1.9	-1.9	-1	0
v_3	-2.71	-1.9	-1	0

For example, $v_3(30) = -1 + 0.9v_2(22) = -2.71$. For this zero-initialized update, v_k is the optimal k-period return with zero value afterward.

Stopping VFI and Recovering a Policy

After computing $v_{k+1} = Tv_k$, measure the **Bellman residual**:

$$d_k = \|Tv_k - v_k\|_\infty = \|v_{k+1} - v_k\|_\infty.$$

Contraction implies

$$\|v_k - V^*\|_\infty \le d_k + \beta\|v_k - V^*\|_\infty, \quad \text{so} \quad \|v_k - V^*\|_\infty \le \frac{d_k}{1-\beta}.$$

- For value error at most ε, stop when $d_k \le (1-\beta)\varepsilon$ and return v_k.
- Recover a policy from the scores computed using this returned value:

$$\widehat{\sigma}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q_k(s,a).$$

- Evaluate $V^{\widehat{\sigma}}$ to check the policy's achieved return. Intermediate greedy policies in VFI need not improve monotonically.

The bound assumes exact expectations and maximization; numerical approximations add errors.

Example: VFI for McCall Job Search

The state is (U, w) or (E, w). Rejecting pays b and draws a new offer w' with probability $q(w')$; accepting pays w forever.

$$v_k((\mathrm{E}, w)) = \frac{w}{1-\beta} \quad \text{for every } k.$$

Use this known employed value and update unemployed states:

$$\begin{aligned} Q_k((\mathrm{U}, w), \text{accept}) &= \frac{w}{1-\beta}, \\ Q_k((\mathrm{U}, w), \text{reject}) &= b + \beta \sum_{w'} q(w') v_k((\mathrm{U}, w')), \\ v_{k+1}((\mathrm{U}, w)) &= \max\{Q_k((\mathrm{U}, w), \text{accept}),\ Q_k((\mathrm{U}, w), \text{reject})\}. \end{aligned}$$

Initialize $v_0((\mathrm{U}, w)) = 0$. Employed-state values stay fixed at their known values; only unemployed-state values need iteration.

Example: McCall Iterations

Use $\beta = 0.9$, $b = 1$, offers $(1, 2, 3, 4, 5)$, and probabilities $(0.1, 0.2, 0.4, 0.2, 0.1)$.

	$w = 1$	$w = 2$	$w = 3$	$w = 4$	$w = 5$
v_0	0.00	0.00	0.00	0.00	0.00
v_1	10.00	20.00	30.00	40.00	50.00
v_2	28.00	28.00	30.00	40.00	50.00
v_3	31.06	31.06	31.06	40.00	50.00
v_{10}	34.20	34.20	34.20	40.00	50.00
V^*	34.32	34.32	34.32	40.00	50.00

The columns give $v_k((\mathrm{U}, w))$. After the first update, the probability-weighted mean is 30. For offer 2, the next update is

$$v_2((\mathrm{U}, 2)) = \max\{20,\ 1 + 0.9 \times 30\} = 28.$$

At convergence, waiting is worth about 34.32: reject offers 1–3 and accept 4–5.

Example: VFI for Continuous Saving

With storage capacity $M \geq 0$, stock is $s \in [0, M + B]$. Choose saving $a \in [0, \min(s, M)]$ and consume $c = s - a$.

- Income $Y' \sim \text{Uniform}[0, B]$, $B > 0$, is independent across periods; next stock is $s' = a + Y'$.
- Reward is $r(s, a) = (s - a)^\alpha$, with $0 < \alpha < 1$.

The update averages over income and maximizes over saving:

$$Q_k(s, a) = (s - a)^\alpha + \frac{\beta}{B} \int_0^B v_k(a + y)\, dy,$$
$$v_{k+1}(s) = \max_{a \in [0, \min(s, M)]} Q_k(s, a).$$

Assume $0 < \beta < 1$. The state and feasible action sets are compact, and rewards are bounded. The exact operator maps continuous functions to continuous functions and is a contraction in the supremum norm.

Computing the Continuous Update

To implement the continuous saving model numerically:

1. **Represent v_k:** store values at stock points and **interpolate** between them.
2. **Compute the expectation:** approximate the income integral using **numerical quadrature** or sampled incomes.
3. **Choose saving:** maximize over the feasible interval using numerical optimization or an action grid.

These approximations add errors beyond the VFI stopping error. Refine the stock grid, integration, and maximization to check that values and decisions are stable.

The model remains continuous. Finite termination of policy iteration for finite MDPs does not automatically apply to this model.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Chapter 3: Finite Markov Decision Processes
   - Chapter 4: Dynamic Programming

2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming — The Bellman Equation and the Principle of Optimality
   - Discrete State Dynamic Programming — Policy Function Iteration
   - Discrete State Dynamic Programming — Value Function Iteration
   - Job Search I: The McCall Search Model — The McCall Model
   - Discrete State Dynamic Programming — Example: A Growth Model
   - The Income Fluctuation Problem I: Basic Model — The Optimal Savings Problem